DATA STRUCTURE AND ALGORITHMS

LESSON 02 - RECURSION

Conceptual Difficulty 

Implementation Difficulty 

LECTURER: SEAK LENG

OVERVIEW

1. Introduction to algorithm
2. Basic data types and statements
3. Control structures and Loop
4. Array
5. Data structure
6. Sub-programs

7. *Recursive*
8. Pointers
9. Linked Lists
10. Stacks and Queues
11. Sorting algorithms
12. Trees

C++

OUTLINE

- Review on Function
- What is recursive function?
- How to use recursive
- Examples and Practices

REVIEW ON FUNCTION

- What is a function? A function is a block of code that performs a specific task. A function may return a value.
- Why function?
  - ✓ A large and complex program can be divided into smaller programs
    - ✓ assign task and work in team is easy
  - ✓ Program is easy to understand, fix error, and maintain
  - ✓ Reusable code
  - ✓ A function can be call again and again anytime and anywhere in the program

REVIEW ON FUNCTION

- There are two types of functions in C programming (likewise, in C++):

1. Standard library functions
   - Also called: built-in function, or existing function
   - For example, in C++, see table 1

Library	Provided functions
iostream	cout, cin
cmath	Sqrt(), pow(), abs()
string	Length(), substr(), find()
fstream	.is_open(), .close(), getline()

2. User-defined functions
   - Also called: user-custom function
   - For example, a user (programmer) defines his/her own function to do something

REVIEW ON FUNCTION

- Create a function with a returning value

```
type functionName(type parameter1){
    ... .. ...
    ... .. ...
    return value;
}
```

a) Type of value returning from the function

d) Return statement
It must be the same data type as a)

c) Parameter of a function
It has parameter type and parameter name
If have more than one, separate them by comma

```
#include <iostream>
using namespace std;
int sumTwoNumber(int n1, int n2){
    int s;
    s=n1+n2;
    return s;
}
int main(){
    sumTwoNumber(3, 5);
}
```

REVIEW ON FUNCTION

- Create a function with no returning value

```
void functionName(type parameter1){
    ... .. ...
    ... .. ...
    return value;
}
```

a) Void means no returning value from the function

d) No need to *return statement*

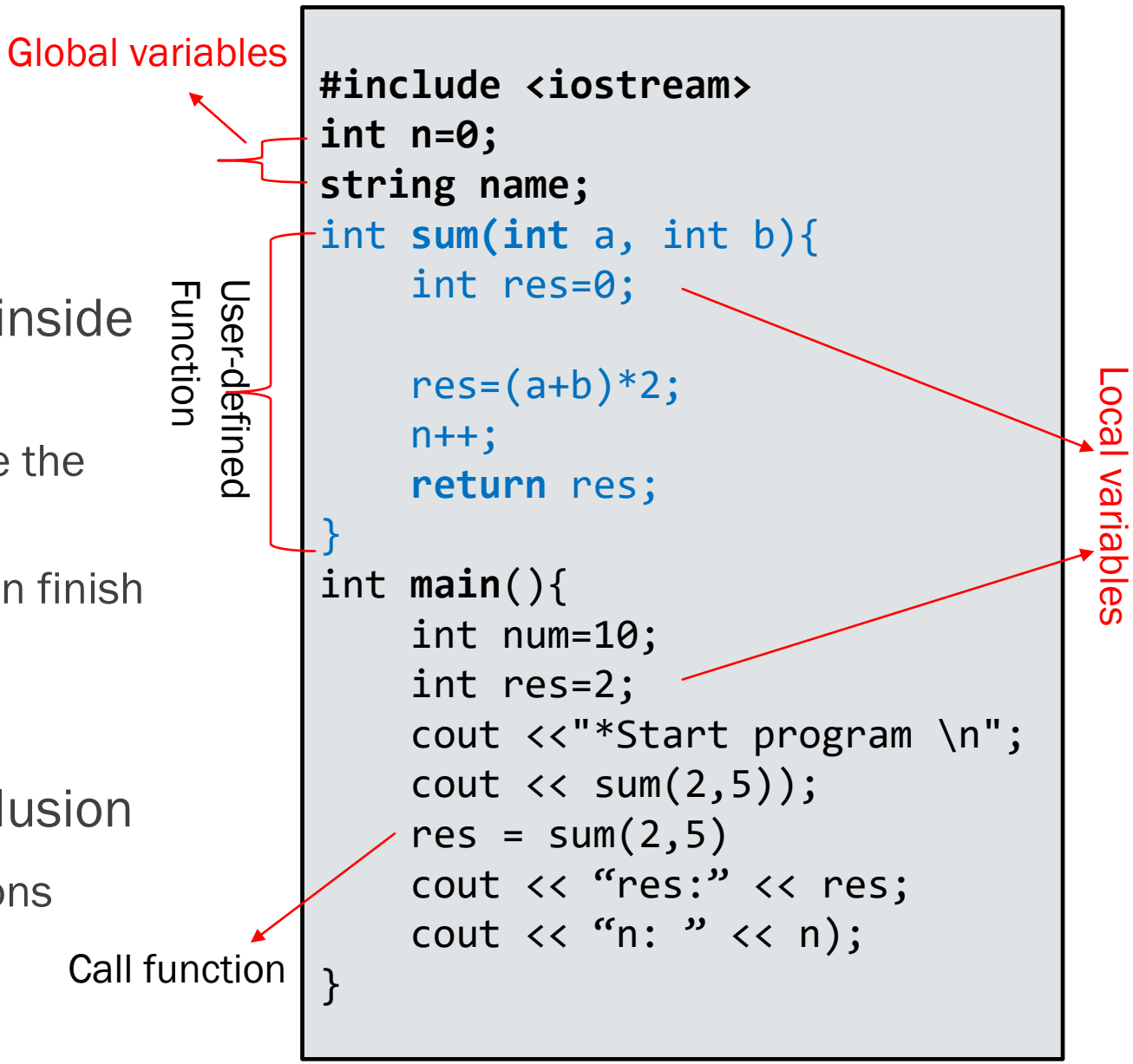
```
#include <iostream>
using namespace std;
void greetMessage(char name[20]){
    cout << "Hi, " << name << endl;
    cout << "Welcome to GIC" << endl;
}

int main(){
    greetMessage("Sok");
}
```

REVIEW ON FUNCTION

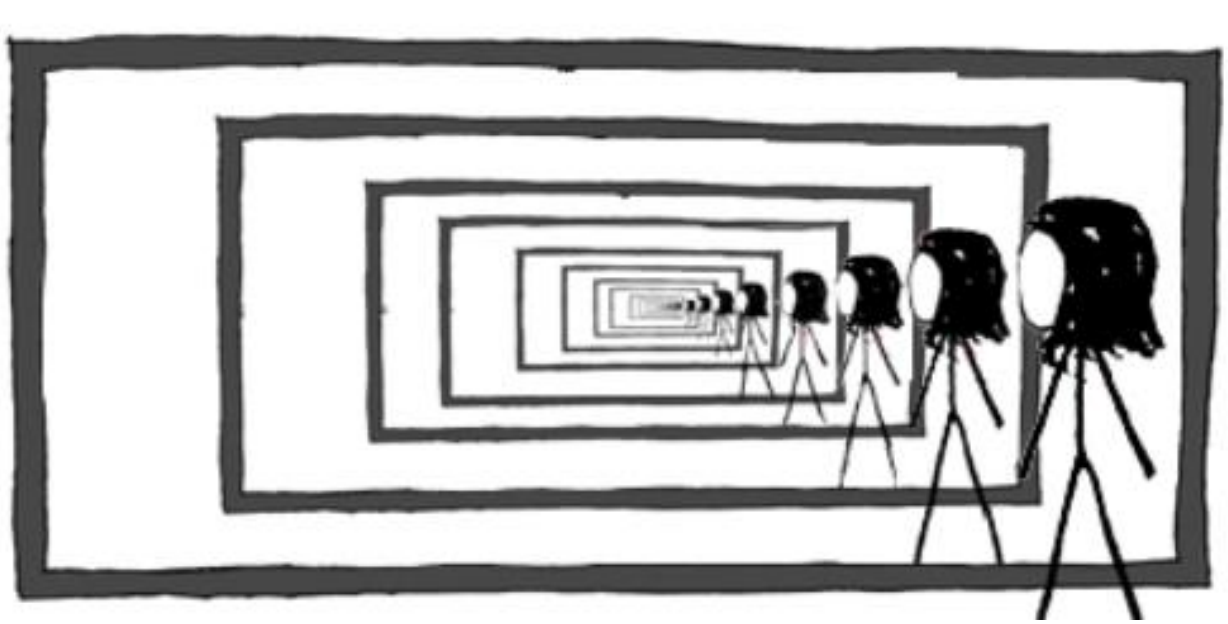
- Global vs Local Variable
  - **Local variable** a variable that creates inside a function
    - ✓ Its value can not be accessed from outside the function
    - ✓ Its value will be destroyed after the function finish executing
  - **Global value** is variable that creates

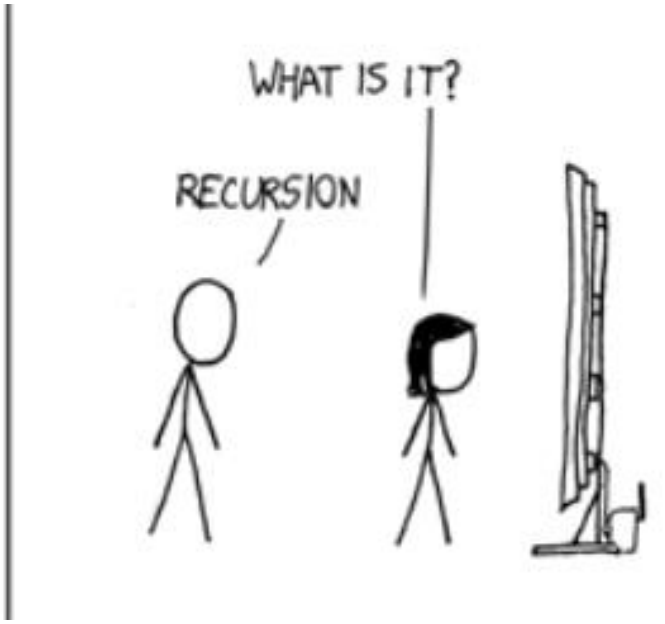
outside the function below all library inclusion

- ✓ Its value can be accessed from any functions
- ✓ Its value will be destroyed after the whole program finish executing



Recursive Function





WHAT IS RECURSION?

- Recursion is a general programming technique where a problem is solved by breaking it down into smaller instances of the same problem.
- Advantages:
  - Clean and less code
  - Useful for sorting and searching algorithm
- Disadvantages:
  - Slower than iterative approach
  - More memory

RECURSIVE FUNCTION

- **Recursive function** is a function that calls itself repetitively until certain condition is satisfied. It works like a loop.

- Component of recursive function

1. Recursive case: when the function should call itself again
2. Base case: when the function should not call itself again

In recursive case, the function calls itself with smaller values until it reaches the base cases.

```
returntype func(){
    //Base case
    // Recursive case
    func();
}
```

HOW IT WORKS

- Example: Factorial Function 5!

5! = 5 X 4 X 3 X 2 X 1

4! = 4 X 3 X 2 X 1

3! = 3 X 2 X 1

2! = 2 X 1

1! = 1

n! = n x (n-1)!
Factorial(n) = n * factorial(n-1)

```
Function fac(n: integer): integer
Begin function
      // Base case
      if(n<=1) then
            return 1
      else // recursive case
            return n*(fac(n-1))
      end if
End function
```

HOW IT WORKS

- Example: Factorial Function 5!

```
Function fac(n: integer): integer
Begin function
      // Base case
      if(n<=1) then
            return 1
      else // Recursive case
            return n*(fac(n-1))
      end if
End function
```

Algorithm for Factorial Function using Recursion

```
int factorial(int n) {
    // Base case: factorial of 0 is 1
    if (n <= 1) { return 1; }
    // Recursive case:  n * factorial(n-1)
    return n * factorial(n - 1);
}
```

Recursive in C++ programming language

HOW IT WORKS

```
int factorial(int n) {
        if (n <= 1) { return 1; }
        return n * factorial(n - 1);
}
```

For factorial(5), the function works like this:

- factorial(5) → 5 * factorial(4)
- factorial(4) → 4 * factorial(3)
- factorial(3) → 3 * factorial(2)
- factorial(2) → 2 * factorial(1)
- factorial(1) → returns 1 (base case)

Now, the recursion starts unwinding:

- factorial(1) → 1
- factorial(2) → 2 * 1 = 2
- factorial(3) → 3 * 2 = 6
- factorial(4) → 4 * 6 = 24
- factorial(5) → 5 * 24 = 120
- So, factorial(5) returns 120.

RECURSIVE FUNCTION

- Example of recursive function in Suit Fibonacci
- The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones. It starts like this: 0,1,1,2,3,5,8,13,21,34,...
- The Fibonacci sequence is defined as:

$$F(n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ F(n-1) + F(n-2), & \text{if } n \geq 2 \end{cases}$$

```
Function fibo(n: integer): integer
Begin function
        if(n==0) then return 0
        if(n==1) then return 1
        else
             return fibo(n-1)+fibo(n-2)
        end if
End function
```

Algorithm for Suit Fibonacci using Recursion

RECURSIVE

$$F(n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ F(n-1) + F(n-2), & \text{if } n \geq 2 \end{cases}$$

- Example of recursive function in Suit Fibonacci

```
Function fibo(n: integer): integer
Begin function
        if(n==0) then return 0
        if(n==1) then return 1
        else
            return fibo(n-1)+fibo(n-2)
        end if
End function
```

Algorithm for Suit Fibonacci using Recursion

```
int fib(int n){
    if(n==0) return 0; //Base case
    if(n==1) return 1; //Base case
    return fib(n-1) + fib(n-2); //Recursive case
}

int main(){
    cout << fib(5) << endl;
    for(int i=0; i<=8; i++){
        cout << fib(i) << " ";
    }
}
```

Output

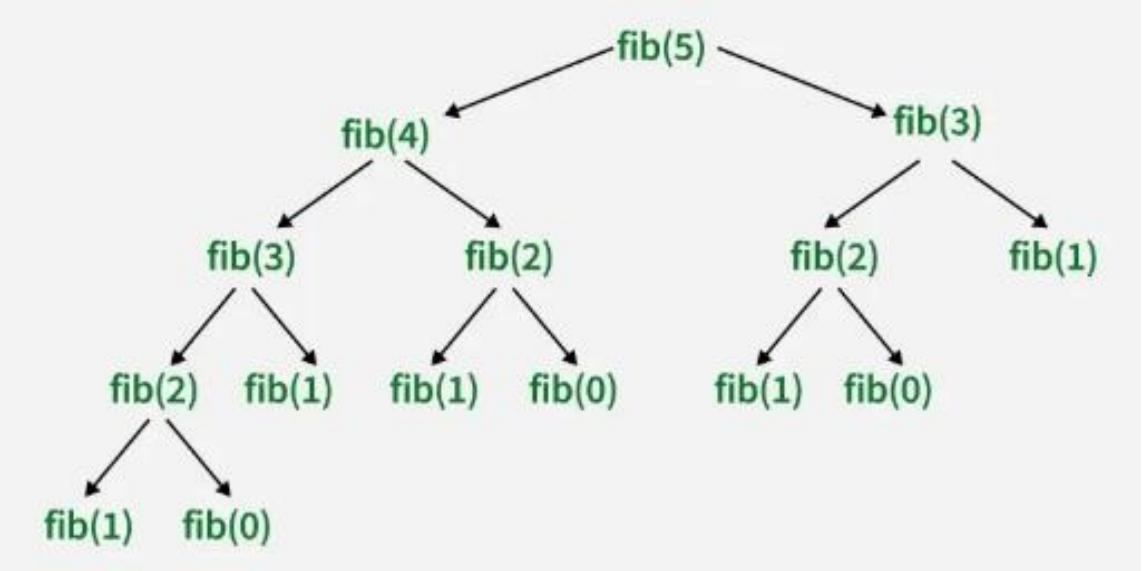
```
5
0 1 1 2 3 5 8 13 21
```

RECURSIVE

$$F(n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ F(n-1) + F(n-2), & \text{if } n \geq 2 \end{cases}$$

- Example of recursive function in Suit Fibonacci

Recursion Tree




```
int fib(int n){
    if(n==0) return 0; //Base case
    if(n==1) return 1; //Base case
    return fib(n-1) + fib(n-2); //Recursive case
}

int main(){
    cout << fib(5) << endl;
    for(int i=0; i<=8; i++){
        cout << fib(i) << " ";
    }
}
```

Output

```
5
0 1 1 2 3 5 8 13 21
```

RECURSIVE: DIRECT AND INDIRECT

1. Direct Recursion: A function is directly recursive when it calls itself within its own body.
2. Indirect Recursion: A function is indirectly recursive when it calls another function, which in turn calls the original function.

Example:

```
f(x)=1                  if x=1 or x=2
f(x)=f(x-1)*f(x-2)      if x>2
```

Direct recursive

```
f(x)=1                  if x=1
f(x)=g(x)+2             if x>1
g(x)=0                  if x=2
g(x)=f(x)-1             if x>2
```

Indirect recursive

RECURSIVE: DIRECT AND INDIRECT

- Example: Direct Recursion

```
int fibonacci(int n){
    if(n==0) {return 0;}
    if(n==1) {return 1;}
    return fibonacci(n-1) + fibonacci(n-2); //Calls itself directly
}

int main(){
    for(int i=0; i<10; i++){
        cout << fibonacci(i) << " ";
    }
}
```

Output

```
0 1 1 2 3 5 8 13 21 34
```

RECURSIVE: DIRECT AND INDIRECT

- Example: Indirect Recursion

```
void odd(int n){
    if(n<=10){
        cout << n << " is odd\n";
        even(n+1);
    }
}
void even(int n){
    if(n<=10){
        cout << n << " is even\n";
        odd(n+1);
    }
}
```

```
int main(){
    int n=1;
    odd(n);
}
```

Output

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
6 is even
7 is odd
8 is even
9 is odd
10 is even
```

PRACTICE

- What is the result of this function?

1

```
void display(int n){
    if(n==0){
        return;
    }else{
        cout << n << " " ;
        display(n-1);
    }
}

int main(){
    display(10);
}
```

2

```
void display(int n){
    if(n==0){
        return;
    }else{
        display(n-1);
        cout << n << " " ;
    }
}

int main(){
    display(10);
}
```

PRACTICE

- What is the result of this function?

3

```
#include <iostream>
using namespace std;

void fun(int n) {
    if (n == 0)
        return;
    fun(n / 2);
    cout << n % 2;
}

int main() {
    fun(10);
    return 0;
}
```

PRACTICE

- What is the result of this function?

4

```
#include <iostream>
using namespace std;

int fun(int n, int &count) {
    count++;
    if (n <= 1)
        return n;
    else
        return fun(n - 1, count) + fun(n - 2, count);
}

int main() {
    int count = 0;
    cout << fun(5, count) << endl;
    cout << "count = " << count << endl;
    return 0;
}
```

PRACTICE

Using recursive function

1. Calculate summation of $S = 1^2+2^2+3^2+...+n^2$,where n is an integer number input by user, using recursive function.
2. Calculate power n of an integer m denoted by m^n , using recursive function

   Hint: $m^n = m*m*....*m$
3. Calculate the multiplication between 2 positive integers using this method:

   $m*n = m+m+....+m$
4. Display n stars (*) where n is a positive integer number input by a user

PRACTICE

Using recursive function

5. Compute the sum of all elements in an array, where the array and its size are given as parameters
6. Find the minimum value of an array
7. Check how many times a character appears in a string
8. Find the sum of digits of a number